

3.1	Creating and Executing Shell Scripts (nano, vi, ./script.sh)
3.2	Shell Metacharacters and Operators
3.2.1	Filename Expansion (wildcards: *, ?, [])
3.2.2	Input/Output Redirection (>, >>, <)
3.2.3	Pipes ()
3.2.4	Command Substitution (\${...}, ...)
3.3	Control Flow Structures (if-else, case, for, while, until)
3.4	Logical Operators (&&, , !)
3.5	test and [] command for Condition Testing (file, numeric, string)
3.6	Arithmetic Operations (expr, \$(()))

3.1 Creating and Executing Shell Scripts (nano, vi, ./script.sh)

1. What is a Shell Script?

A **shell script** is a text file containing a series of commands that the Linux shell (like bash) can execute.

- File extension: .sh (by convention, not mandatory)
 - Interpreter: Usually bash or sh
-

2. Tools Required

- **Linux terminal**
- **Nano text editor** (comes pre-installed in most Linux distributions)

To check if Nano is installed:

```
nano --version
```

3. Creating a Shell Script Using Nano

Step-by-Step Process:

1. **Open Terminal**
2. **Create a new shell script file using nano**
Example: myscript.sh

```
$ nano myscript.sh
```

3. Write your shell script code

Here's a sample shell script:

```
#!/bin/bash
echo "Hello, World!"
date
```

- `#!/bin/bash` is called the hashbang. It tells the system to use the bash shell to interpret the script.

4. Save the file

- Press Ctrl + O (write out)
- Press Enter to confirm the filename
- Press Ctrl + X to exit nano

4. Make the Script Executable

Before executing, you need to give the script execute permission.

```
$ chmod +x myscript.sh
```

5. Run (Execute) the Shell Script

There are two ways to run it:

1. Using relative path:

```
$ ./myscript.sh
```

2. Using bash directly:

```
$ bash myscript.sh
```

6. Example: A Complete Session

```
$ nano greet.sh
Script content:
#!/bin/bash
echo "Welcome to Shell Scripting"
echo "Today's date is: $(date)"
Save and exit.
```

Then:

```
$ chmod +x greet.sh
```

```
$ ./greet.sh
```

Output:

Welcome to Shell Scripting

Today's date is: Mon Jul 14 10:00:00 UTC 2025

7. Tips for Beginners

- Always use `#!/bin/bash` at the top of the script.
 - Use comments `#` to describe each step in your script.
 - Test your script with `bash scriptname.sh` if execution fails.
-

8. Practice Exercises

1. Create a script that asks for your name and prints a greeting.
2. Write a shell script that prints the following:
 1. Current date and time
 2. Your username
 3. System uptime
 4. Current working directory
3. Write a script that:
 1. Prompts the user to enter a file name
 2. Checks if the file exists in the current directory
 3. Prints whether it exists or does not exist
4. Create a script that:
 1. Prompts the user to enter two numbers and an operation (+, -, *, /)
 2. Performs the calculation and prints the result
5. Create a script that:
 1. Asks the user to enter a number
 2. Checks if the number is even or odd
 3. Prints a message accordingly
6. Write a script that:
 1. Asks the user to enter a number N
 2. Prints all numbers from 1 to N using a loop

Summary

Task	Command / Description
Open file in nano	<code>nano filename.sh</code>
Make script executable	<code>chmod +x filename.sh</code>
Run the script	<code>./filename.sh</code> or <code>bash filename.sh</code>
Save in nano	<code>Ctrl + O, Enter</code>
Exit nano	<code>Ctrl + X</code>

Steps to Create and Edit a Shell Script with vi

1. Open a new file in vi

Type this in the terminal:

```
vi myscript.sh
```

This opens vi and creates a file named myscript.sh (if it doesn't exist).

2. Switch to insert mode to start typing

When vi opens, you're in **command mode** (you can't type text yet).

Press: `i` (insert mode)

This switches vi into **insert mode** so you can start typing.

You'll see -- INSERT -- at the bottom of the screen.

3. Write your shell script

Now type your shell script. For example:

```
bash
CopyEdit
#!/bin/bash
# This script prints system hostname

echo "Your system hostname is: $(hostname)"
```

4. Save the file and exit vi

After typing your script:

1. Press **Esc** to leave insert mode (back to command mode).
2. Type `:w q`

and press **Enter**.

- `:w` = write (save the file)
- `:q` = quit (exit vi)

3.2 Shell Metacharacters and Operators

1. Introduction

In Linux, the **shell** acts as a command-line interpreter that takes user input and executes commands. To make shell commands powerful and flexible, it provides **metacharacters** and **operators** which have special meanings.

These symbols help in **file manipulation**, **process control**, **redirection**, **pattern matching**, and **conditional execution**.

2. What are Shell Metacharacters?

Metacharacters are special characters in Linux shells (like Bash, Zsh, etc.) that are interpreted by the shell to perform specific functions instead of being treated as literal text.

Common Metacharacters:

Metacharacter	Description	Example
*	Matches zero or more characters (wildcard)	<code>ls *.txt</code> (lists all .txt files)
?	Matches any single character	<code>ls file?.txt</code> (matches file1.txt, fileA.txt)
[]	Matches any one character in the brackets	<code>ls file[123].txt</code> (matches file1.txt, file2.txt, file3.txt)
{}	Brace expansion for multiple patterns	<code>echo {a,b,c}</code> → a b c
~	Refers to the home directory of the user	<code>cd ~</code> (changes to home dir)
\	Escape character to suppress special meaning	<code>echo \HOME</code> (prints HOME literally)

Metacharacter	Description	Example
' '	Single quotes to prevent interpretation of metacharacters	echo '\$HOME'
" "	Double quotes to suppress interpretation except for \$, `, and \	echo "\$HOME"
`command`	Command substitution (old style)	echo `date`
\$(command)	Command substitution (modern style)	echo \$(date)
#	Comment in scripts or CLI	# This is a comment

3. Shell Operators

Operators are symbols or keywords that let you perform operations like redirection, piping, conditional execution, and arithmetic.

3.1 Redirection Operators

Operator	Description	Example
>	Redirects output to a file (overwrite)	echo "Hello" > file.txt
>>	Appends output to a file	echo "World" >> file.txt
<	Takes input from a file	wc -l < file.txt
2>	Redirects error output (stderr)	ls /fake_dir 2> error.log

3.2 Pipe Operator (|)

Linux Piping Mechanism

In Linux, **pipes (|)** are used to pass the **output of one command** as the **input to another command**. This allows multiple commands to be combined in a **pipeline**, enabling powerful and efficient data processing directly from the shell.

The pipe operator | connects the **stdout (standard output)** of the first command to the **stdin (standard input)** of the second command.

Key Features:

- Avoids creating temporary files.
- Reduces I/O overhead by directly streaming data between processes.
- Supports chaining multiple commands.

Examples:

1. **Display logged-in users and count them:**

```
who | wc -l
```

- who: Lists logged-in users.
- wc -l: Counts the number of lines (users).

2. **Find number of files in a directory:**

```
ls -l | wc -l
```

- ls -l: Lists files one per line.
- wc -l: Counts the lines (files).

3. **Search for a pattern in a file and sort it:**

```
cat file.txt | grep "error" | sort
```

- cat file.txt: Displays file content.
- grep "error": Filters lines containing "error".
- sort: Sorts the filtered output.

3.3 Logical Operators

Logical operators in Linux are used to combine multiple commands and control their execution based on success or failure. The most common logical operators are:

1. **AND (&& or -a)**

Executes the second command **only if** the first command succeeds (returns exit status 0).

Example:

```
$ mkdir test && cd test
```

cd test runs only if mkdir test is successful.

2. **OR (|| or -o)**

Executes the second command **only if** the first command fails (non-zero exit status).

Example:

```
$ cd test || echo "Directory not found"
```

Prints a message if cd test fails.

3. NOT (!)

Inverts the exit status of a command.

Example:

```
$ if [ ! ls file.txt ]
```

Returns success if ls file.txt fails.

```
if [[ $var1 -gt 5 && $var2 -lt 10 ]]; then ... fi
if [[ $var1 -eq 0 || $var2 -eq 0 ]]; then ... fi
if [[ ! -f "myfile.txt" ]]; then echo "File does not exist."; fi
```

3.4 Background and Control Operators

Operator	Description	Example
&	Runs command in background	./Longtask.sh &
;	Executes commands sequentially	echo "one"; echo "two"

3.5 Arithmetic Operators (used with expr or \$(()))

Operator	Description	Example
+	Addition	expr 5 + 3 or echo \$((5+3))
-	Subtraction	expr 5 - 2
*	Multiplication	expr 4 * 2 (* is metachar so required escaping)
/	Division	expr 8 / 2
%	Modulus	expr 7 % 3

4. Examples Combining Metacharacters and Operators

- Find and count .txt files:**
ls *.txt | wc -l
- Run two commands only if the first succeeds:**
./myscript.sh && echo "Script ran successfully"
- Run a command in the background:**
tar -czf backup.tar.gz /home/user &
- Command Substitution**
x=`wc -l < vmpfile1.txt`

5. Arithmetic equation evaluation with command substitution

```
$ x=10
$ y=20
$ z=$((x + $y))
$ echo $z
30
6. $ echo "Today is $(date)"
7. echo "Number of files: $(ls -l $(pwd) | wc -l)"
8. current_date=$(date)
   echo "The date is $current_date"
9. echo "There are $(ls | wc -l) files in the current directory"
10. #!/bin/bash
    user=$(whoami)
    echo "Hello, $user! Welcome back."
```

3.3 Control Flow Structures (if-else, case, for, while, until)

- Control structure alters the flow of execution of shell script.
- The UNIX supports two types of control structure
 1. Decision making control structure and
 2. Loop control structure
- Decision making control structure allows the computer to take the decision depending upon the condition executes the statement(s) or skips some statements.
- The Loop control structure helps computer to execute a group of statements repeatedly for specified number of times.

1. Decision making control structure

- The shell offers two decision making control structure:
 - (a) if statement
 - (b) case-esac statement

(a) if statement

- It takes decisions, depending on of certain condition.
- There are three types of if statement

- i. if-then-fi statement
- ii. if-then-else-fi statement
- iii. if-then-elif-else-fi statement

i. if-then-fi statement:

Syntax:

```

if control command
then
    statement(s)
fi

```

- Where **if**, **then** and **fi** are keywords.
- A control command following **if** is any valid UNIX command.
- If the exit status of this control command is zero (successful) then the statement(s) between the **then** and the **fi** are executed otherwise statement(s) are skipped.

Example:

```

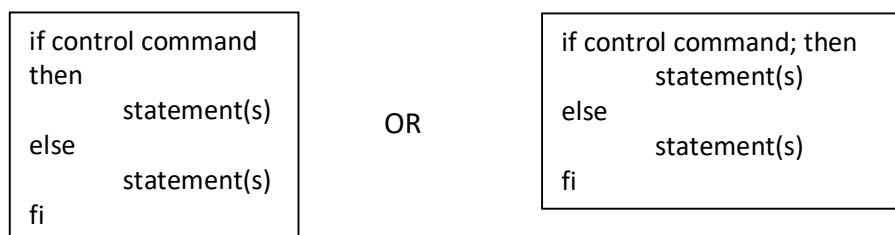
x=5
y=10
if test $x -lt $y
then
    echo "$x is less than $y"
fi

```

ii if-then-else-fi statement

- It is a two-way decision making statement.

Syntax:



Here if, then, else and fi are keywords. If the exit status of a control command is zero then it executes the statements between then and else otherwise executes the statements between else and fi.

Example:

```

x=5;y=10
if test $x -gt $y; then
    echo "$x is larger"
else
    echo "$y is larger"
fi

```

iii if-then-elif-else-fi statement:

- This is the third form of if construct. A user can use this construct instead of multilevel if-then-else statements.
- The advantage of this form over multiple if is that, if-elif ends with only single fi whereas in multiple if construct, fi write number of if times.

```

if control command1
then
    statement(s)
elif control command2
then
    statement(s)
else
    statement(s)
fi

```

Example:

```

read -p "Enter a number" num
if [ num -gt 0 ]; then
    echo "number is positive"
elif [ num -lt 0 ]; then
    echo "number is negative"
else
    echo "number is zero"
fi

```

(b) The case statement

- The case statement is a better organizational tool than a multilevel if-then-else-fi statement. The case statement will enable you to match several values against one variable. It is useful in menu-driven script.

Syntax:

```

Case expression/variable in
    label1)
        statement(s)
        ;;
    label2)
        statement(s)
        ;;
    labelN)
        statement(s)
        ;;
    [*])
        statement(s)
        [;:]
esac

```

- Here **case**, **in** and **esac** are keywords.
- The value of expression/variable is compared against the labels until a match is found.
- When a match is found, the shell executes all statements within that case until pair of semicolons (i.e.;;) is encountered.
- The last label *) represents the default clause and gets executed when all other cases fail.
- The label enclosed within square bracket indicates optional part.
- label1, label2...labelN are different options.
- It may include:
 - String or character
 - One or more string/character separated by pipe (|)
 - Pattern matching characters like *,? and [...]

Example-1: The following code accepts a number between 1 and 9 and display inputted number in word.

```
echo "Enter No:"; read num
case $num in
  1) echo "One"
    ;;
  2) echo "Two"
    ;;
  3) echo "Three"
    ;;
  4) echo "Four"
    ;;
  5) echo "Five"
    ;;
  6) echo "Six"
    ;;
  7) echo "Seven"
    ;;
  8) echo "Eight"
    ;;
  9) echo "Nine"
    ;;
  *) echo "Please, enter a number between 1 to 9"
    ;;
esac
```

Example-2: A user can use pattern matching characters in label. For example, the following code checks whether an inputted character is alphabet, digit or special character.

```
Read -p "Enter any character" ch
case $ch in
```

```

[0-9]) echo "You entered a digit"
;;
[a-zA-Z]) echo "You entered an alphabet"
;;
*) echo "You entered a special character"
esac

```

2. Loop control structure

- A loop involves repeating some portion of the program either a specified number of times or until a particular condition is being satisfied.
- There are three methods by which we can repeat a part of program.
- They are
 - i. while loop
 - ii. until loop
 - iii. for loop

i. while loop:

- To repeat a part of program fixed number of times then while loop is used.

Syntax:

```

while control-command
do
    statement(s)
done

```

- Here while, do and done are keywords.
- The loop will execute the statement(s) repeatedly till the exit status of control-command is zero (true) otherwise control transfer to next statement following the done.

Example:

You can display 1 to 10 as follows.

```

i=1
while [ $i -le 10 ]
do
    echo $i
    i=$(( $i + 1 ))    #or i=`expr $i + 1`
done

```

ii. until loop

It is a complement loop of while that means until loop is executed till the exit status of control command is non zero (false) otherwise control is transferred to the next statement following the done.

Syntax:

```
until control-command
do
    statement(s)
done
```

- Here **until**, **do** and **done** are keywords.
- There is a slight difference between the working of while and until loops.
- The while loop executes till the exit status of the control-command is true (0) and terminates when the exit status becomes false (non-zero) whereas until loop executes till the exit status of the control command is false (non-zero) and terminates when this status becomes true (0).

Example:

You can display 1 to 10 as follows;

```
i=1
until [$i -ge 10 ]
do
    echo $i
    i=$((i + 1))    #or i=`expr $i + 1`
done
```

iii. **for loop**

- Unlike while and until, for does not test exit status of control statement.
- The working of this loop is different than the other two loops.

Syntax:

```
for control-variable in value1 value2....valueN
do
    statement(s)
done
```

- Here for, in, do and done are keywords.
- It allows us to specify a list of value after in keyword; each value is separated by white space character.
- The control variable in the loop can take these values and execute the statements within the loop until no value is left.

Example-1: Display 1 to 5

```
for i in 1 2 3 4 5
do
```

```
    echo $i
done
```

Example-2: Display all command line arguments.

```
echo "This program displays all cmd line arguments"
for x in $@
do
    echo $x
done
```

NOTE: for loop in UNIX shell script also supports C programming structure of for loop as shown in below example.

Example: Print 1 to 10.

```
for (( i=1; i<=10; i++))
do
    echo $i
done
```

3.4 Logical Operators (&&, ||, !)

Covered under topic 3.2

3.5 test and [] command for Condition Testing (file, numeric, string)

- It is used to evaluate expressions and returns either true or false exit status which is used by **if** statement to make decisions.
- There are two major functionalities of **test** statement.
 - i. Compares two numbers.
 - ii. Compares two strings or a single one for a null value.
 - iii. Checks a file's attributes.

i. Numeric comparison

- It performs numeric comparison between two numbers.
- Table shows the list of operators used to perform numeric comparison between the numbers.
- Each operator begins with a hyphen (i.e. -) followed by two character word.
- Either side of the operator must be enclosed with white space character.

List of operators

Operator	Meaning
-eq =	equal to
-ne !=	Not equal to
-gt >	Greater than
-ge >=	Greater than equal to
-lt <	Less than
-le <=	Less than equal to

ii. String comparison

- The test statement also handles string comparison.
- It uses operators to compare a string are shown in table.

List of string comparison operators

Operator	Meaning
str1=str2	It returns true if the strings str1 and str2 are same.
str1 != str2	str1 - str2 It returns true if the strings str1 and str2 are different.
-n str	It returns true if the length of string str is greater than 0 (zero). i.e. str is not a null string.
-z str	It returns true if the length of string str is zero i.e. null string
str	It returns true if string str is assigned and not null.
str1==str2	It returns true if the strings str1 and str2 are same.

iii File Tests

- test can be used to test the various file attributes like its type (file, directory or symbolic link) or its permissions (read, write, execute, SUID, etc.).

Test	True if file
-f file	file exists and is a regular file
-r file	file exists and is readable
-w file	file exists and is writable
-x file	file exists and is executable
-d file	file exists and is a directory
-s file	file exists and has a size greater than zero
-e file	file exists (Korn and Bash only)
-u file	file exists and has SUID bit set
-k file	file exists and has sticky bit set
-L file	file exists and is a symbolic link (Korn and Bash only)

f1 -nt f2	f1 is newer than f2 (Korn and Bash only)
f1 -ot f2	f1 is older than f2 (Korn and Bash only)
f1 -ef f2	f1 is linked to f2 (Korn and Bash only)

3.6 Arithmetic Operations (expr, \$(()))

In this section, we will discuss some of the UNIX commands that are used to evaluate arithmetic expression.

1. expr

- This command has two functions: It performs arithmetic operations on integers and string manipulation on strings.

Syntax:

expr [EXPRESSION]

- It displays the value of EXPRESSION onto the standard output.
- Table shows the arithmetic EXPRESSION used with expr command.

Expression	Meaning
ARG1+ARG2	It returns arithmetic sum of ARG1 and ARG2.
ARG1-ARG2	It returns arithmetic difference of ARG1 and ARG2.
ARG1*ARG2	It returns arithmetic product of ARG1 and ARG2.
ARG1/ARG2	It returns arithmetic quotient of ARG1 divided by ARG2.
ARG1%ARG2	It returns arithmetic remainder of ARG1 divided by ARG2.

2. bc (Basic Calculator)

bc command is used for command line calculator. It is similar to basic calculator by using which we can do basic mathematical calculations.

Linux or UNIX operating system provides **the bc command and expr command** for doing arithmetic calculations. You can use these commands in bash or shell script also for evaluating **arithmetic expressions**.

Examples:

Input : \$ echo "12+5" | bc

Output : 17

Input : \$ echo "10^2" | bc

Output : 100

Input: `$ x=`echo "12+5" | bc` ;
echo $x
Output: 17`

Relational operators (>,<,>=,<=,==,!=) are supported by bc.

Input: `$ echo "10>5" | bc
Output: 1`

Input: `$ echo "1==2" | bc
Output: 0`

Logical or Boolean Operators

Input: `$ echo "10 && 5" | bc
Output: 1`

Input: `$ echo "0 || 0" | bc
Output: 0`

Input: `$ echo "! 0" | bc
Output: 1`

3. Using `$((...))` (in bash)

- In bash `$()` is used for command substitution
- And `(..)` is used for writing arithmetic expression

Syntax: `$(( expression ))`

e.g.: `z=$(( $x + $y ))`

4. factor

- This is another math-related command available in UNIX.
- It also works on either interactive or command line mode.

Syntax:
`factor [NUMBER(S)]...`

Command line argument processing

The following table shows a number of **special variables or parameters** that you can use in your shell scripts for processing command line arguments.

Sr. No	Variable & Description
1.	\$0 The filename of the current script.
2.	\$n or \$1,\$2,\$3.... g These variables correspond to the arguments with which a script was invoked. Here n is a positive decimal number corresponding to the position of an argument (the first argument is \$1, the second argument is \$2, and so on). Or Positional parameters
3.	\$# The number of arguments supplied to a script.
4.	\$* All the arguments are double quoted. If a script receives two arguments, \$* is equivalent to \$1 \$2. "\$1 \$2"
5.	\$@ All the arguments are individually double quoted. If a script receives two arguments, \$@ is equivalent to \$1 \$2. "\$1" "\$2"
6.	\$? The exit status of the last command executed. (0 if successful and 1 if unsuccessful)
7.	\$\$ The process number of the current shell. For shell scripts, this is the process ID under which they are executing.
8.	\$! The process number of the last background command.

Example:

```
echo "File name: $0"
echo "First Parameter: $1"
echo "Secon Parameter: $2"
echo "All arguments: $@"
echo "All arguments: $*"
echo "Total arguments: $#"
```